

DeepSeek Harness

白皮书

从 0 到 1 玩转 DeepSeek Harness 的新手百科全书

版本 0.1.0-rc.6

2026-08-13

基于 dsh 0.1.0-rc.6 / DeepSeek-V4-Flash-0731

目录

学习路径：从 0 到 1 的 3 天计划

承诺兑现：**任何开发者，3 天从零到能开发插件**。本路径把 9 章拆成 3 天，每天有明确目标与验收标准。

Day 1：理解 + 跑起来（第 1-2 章）

目标：理解 dsh 是什么，能独立启动、对话、配置。

时间	任务	验收
30 分钟	读第 1 章（三个直觉 + 能力矩阵）	能向别人解释"dsh 和 Claude Code 的区别"
30 分钟	第 2 章动手：dsh web 对话 + headless	能启动 UI 完成一次对话；能跑一条 headless 命令
30 分钟	配置模型/推理档位 + 排障速查	能改 settings.yaml；能解决端口占用

验收：dsh --profile headless "你好" 有回复。

Day 2：理解骨架 + 第一个插件（第 3-4 章）

目标：理解 profile/插件机制，写出第一个能跑的插件。

时间	任务	验收
1 小时	第 3 章（profile、挂载、host/client、扩展点、坑）	能挂载社区插件（Git 面板）
1.5 小时	第 4 章抄写 tool-turbo：纯函数 + agent/request 注入	插件加载无错，日志出现决策输出

验收：自己的插件在 dsh 里生效（日志证明注入发生）。

Day 3：实战 + 调优（第 5-6 章）

目标：能评估插件质量、调优性能、理解生态玩法。

时间	任务	验收
1 小时	第 5 章三个真实 PR（安全红线、沙箱约束、测试方法）	能说出"写一个 PR 的完整闭环"
1 小时	第 6 章性能模型（思考占 90%）+ 档位策略 + 坑	能给一个任务选档位、预估瓶颈
30 分钟	第 8-9 章扫读（工具/上下文、MCP/子代理/工作流）+ 术语表	知道 dsh 的能力全貌与扩展方向

验收：能给 dsh 生态贡献一个小改进或写一篇使用心得。

之后：按需深入

想做什么	去读
接外部工具	第 9 章 MCP + 官方 mcp 包文档
并行任务	第 9 章子代理
确定性流程	第 9 章工作流 + 官方 examples
生产部署	官方架构文档 + 安全模型（sandbox/权限）
性能极致	第 6 章 + dsh-tool-turbo 源码
生态变现	第 7 章 + dsh-plugin topic

学习原则（白皮书全程）

第 1 章：认识 DeepSeek Harness

本章目标：从一个完全零基础的角度，建立对 DeepSeek Harness (dsh) 的完整认知——**它是什么、为什么值得学、和主流 Agent 有什么区别、什么时候用它**。读完本章，你不需要任何前置知识，就能理解 dsh 在整个 AI 开发工具版图中的位置。

TL;DR (本章核心, 30 秒版)

1.1 先建立三个直觉

在讲技术之前，先用三个类比建立直觉：

- ① **dsh 是什么？——它是"Agent 的乐高底座"**。想象乐高：官方提供底板和标准积木（运行时 + 核心插件），你可以自由拼装（加插件、换界面、改行为）。对比之下，Claude Code 更像"一辆整车"——很好开，但你想改装发动机得找官方。
- ② **为什么需要"harness"这个词？——它是"套在模型外面的工程层"**。一个模型（DeepSeek V4）本身只会"回复文字"。要让它在你的代码仓库里干活（读文件、跑命令、改代码、多轮循环），外面需要一层工程：会话管理、工具调用、上下文控制、错误恢复。**这层工程就叫 harness**。dsh 是 DeepSeek 官方把这层工程开源出来的产品。
- ③ **为什么 2026 年才开源？——因为 Agent 进入"可编程时代"**。2025 年是"模型能力竞赛"（谁能生成更好的代码）；2026 年进入"Agent 工程竞赛"（谁能更好地组织模型干活）。DeepSeek 开源 harness 的战略意图：**把"如何组织 Agent"这件事变成开放生态**——像当年 Android 开源改变手机生态一样。

1.2 官方定义与核心事实

一句话定义：DeepSeek Harness (dsh) 是 DeepSeek 官方开源的 Agent 运行时，采用"一切皆插件" (everything is a plugin) 架构，基于 Cordis 插件容器构建。

事实	内容
----	----

开源时间	2026-08-13
协议	MIT（可商用、可改）
语言	TypeScript（Node.js ≥ 22）
版本线	0.1.0-rc.x（当前 rc.6，迭代快，官方明示"将有破坏性变更"）
底层	[Cordis](https://github.com/cordiverse/cordis)（可组合插件容器）
内置形态	web（Web UI） + headless（一次性 CLI）
官方定位	官方 README 原话："everything is a plugin"

1.3 架构是怎么"一切皆插件"的

```
flowchart TB
    subgraph Profile["你的 profile（可启动形态）"]
        P1["dsh web（Web UI）"]
        P2["dsh headless（CLI）"]
        P3["自定义 profile（TUI/桌面/机器人...）"]
    end
    end
    subgraph Plugins["能力层（每个能力 = 一个插件）"]
        L["llm：模型接入 + 推理档位"]
        T["tools：写文件/终端/搜索/技能"]
        S["session：会话持久化"]
        C["client：界面（web/终端）"]
        ST["settings：用户配置"]
    end
    end
    subgraph Cordis["Cordis 插件容器"]
        D["依赖注入 · 事件 · 生命周期"]
    end
    end
    Profile --> Cordis
    Cordis --> Plugins
    Plugins --> L
    Plugins --> T
    Plugins --> S
    Plugins --> C
    Plugins --> ST
```

分层视图



= bundle 栈 + 你的补丁层 • dsh web (Web UI 形态) • dsh headless (CLI 形态) • 你的自定义 profile (TUI/桌面/机器人...)	
能力层 (每个能力 = 一个插件) • llm: 模型接入 (DeepSeek V4 系 + 推理档位) • tools: 工具 (写文件/终端/搜索/技能...) • session: 会话持久化 • client: 界面 (web 浏览器半 / 终端半) • settings: 用户配置 ... (60+ 官方包)	
Cordis 插件容器: 加载、依赖注入、事件、生命周期	

三个必须懂的概念

① **profile (可启动形态)** 一个 profile = \$DSH_HOME/profiles/<name>/ 目录, 包含:

② **host 半 / client 半 (一个插件, 两副面孔)**

半边	跑在哪	干什么
host 半	Node 进程	工具、服务、事件、文件系统 ——apply(ctx) 注册
client 半	浏览器 (web profile)	UI、交互——package.json 的 dsh.client 声明

一个 npm 包可同时携带两半 (exports["."] + exports["./client"]) 。

③ **扩展点 (extension point)** 官方原则: "Plugins, not loop changes"——改行为优先
用官方钩子, 不要 fork 核心。常用扩展点 (后续章节逐一实战) :

1.4 DSH 与主流 Agent 的全面对比

能力矩阵 (核心六家)

维度	**dsh**	Claude Code	OpenAI Codex	OpenCode	Gemini CLI	Kimi CLI
开源	✔ MIT	✘ 闭源	✘ 闭源	✔ MIT	✘ 闭源	✘ 闭源

模型绑定	模型无关 (官方适配 DeepSeek)	Claude 系	GPT 系	任意	Gemini 系	Kimi 系
官方运行时	✅ (web + headless + 插件生态)	产品即运行时	产品即运行时	客户端 (无官方后端)	产品即运行时	产品即运行时
插件体系	官方级：一切皆插件，60+ 官方包	配置/钩子为主	配置为主	配置为主	无	无
自定义界面	✅ (client 半 = 自由 UI)	❌	❌	部分 (TUI 固定)	❌	❌
自动化/CI	✅ headless profile	✅	✅	✅	✅	✅
TUI	插件可做 (官方未内置)	✅ 内置	✅ 内置	✅ 内置	✅	✅
生态阶段	零日起步 (2026-08-13)	成熟	成熟	成熟	成熟	早期
适合谁	想深度定制 + 玩生态的开发者	开箱即用	开箱即用	熟悉 OpenCode 用户	Google 生态	Kimi 生态

更多主流 Agent 速览 (一句话定位)

Agent	一句话定位	与 dsh 的核心差异
Cursor	IDE 内嵌的 AI 编码助手 (Composer/Agent 模式)	深度绑定 IDE; dsh 是独立运行时, 可配任意编辑器/终端
Amp	终端原生、代理优先的 AI 编码 agent	轻量终端形态; dsh 多了官方后端 + 插件生态
Devin	云端"AI 软件工程师" (独立任务/浏览器/工作区)	托管云端; dsh 本地运行、数据不出本机

Windsurf	IDE 内嵌编码 agent (Flow/Agent 模式)	同 Cursor, 绑定 IDE
Aider	开源、Git 优先的终端 pair-programming agent	专注"改代码 + git"; dsh 是通用运行时
Qwen Code	阿里通义 coding agent CLI (内置 DeepSeek provider)	主打 Qwen 模型; dsh 官方适配 DeepSeek 且插件化
GLM CLI	智谱 coding agent CLI	主打 GLM 模型; dsh 模型无关 + 插件生态
Grok CLI	xAI 终端 coding agent	主打 Grok 模型

结论：主流 agent 大致分三类——**IDE 内嵌**（Cursor/Windsurf）、**终端编码助手**（Codex/OpenCode/Aider/Qwen Code/GLM CLI/Grok CLI）、**运行时/平台**（dsh/Devin）。dsh 是目前唯一"官方开源 + 模型无关 + 插件生态"的运行型选手。

通俗文字版：每家是什么、适合谁、和 dsh 差在哪

Claude Code (Anthropic)：目前最成熟的终端编码助手。开箱即用、TUI 体验好、生态成熟——**适合想马上干活的人**。缺点：闭源、绑定 Claude 模型、定制空间有限（只能配置/钩子）。和 dsh 比：dsh 能改的东西它改不了（界面/工具链/后端），但 dsh 的"开箱即用"还比不上它。

OpenAI Codex：OpenAI 的终端 agent。工程能力强、GPT 系模型加持。同样闭源绑定。和 dsh 比：能力线接近，但 dsh 开源可自改。

OpenCode：开源、终端、可配任意模型——和 dsh 最像的"邻居"。关键差异：**OpenCode 没有官方后端运行时**（它是客户端 + 配置），dsh 有官方 bundle + 60+ 包 + 插件生态，可定制面更深。**如果你是 OpenCode 用户，迁移 dsh 的成本很低**（概念类似）。

Gemini CLI：Google 的终端 agent。长上下文/多模态是强项（Gemini 模型优势）。绑定 Google 生态。

Kimi CLI：月之暗面的终端 agent。中文场景好、Kimi 模型加持。生态早期。

Cursor / Windsurf: IDE 内嵌型——它们的优势是"编辑器里就用"，劣势是**你被锁在 IDE 里**。dsh 是独立运行时，可以在任意环境（终端/CI/服务器/未来的 TUI）用同一套 agent 能力。

Devin: 云端工程师——你给它任务，它在云上干活。优势是托管、有浏览器；劣势是**数据出本机**、费用高。dsh 本地运行，隐私可控。

Aider: 老牌开源、Git 优先、轻量。适合"只想让 AI 帮改代码"的极简主义者。dsh 是更重的运行时——如果你只需要 Aider 做的事，Aider 够用；如果你想做 Agent 系统，dsh 是底座。

Qwen Code / GLM CLI / Grok CLI: 各家模型的终端 agent——模型绑定是它们的天然属性。dsh 模型无关（官方适配 DeepSeek，可接 OpenAI 兼容），更适合想"模型可换"的人。

一张图理解 dsh 的差异化位置



案例对比：同一个任务，不同的打开方式

任务：让 Agent 在仓库里"找到所有调用某个函数的地方，并统一改一个参数"。

Agent	你会怎么做	体验
dsh (web)	dsh web → 输入指令 → 模型用 Grep/Read/Edit 工具完成	Web UI + 右侧插件侧边栏（可加 Git 面板）
dsh (headless)	dsh --profile headless "任务" → 打印结果退出	可进 CI： 非零退出码即失败
Claude Code / Codex / OpenCode / Kimi	打开 TUI → 输入指令 → 模型完成	终端 TUI，开箱即用

Cursor / Windsurf	在 IDE 里选中代码 → 输入指令	IDE 内嵌体验
Devin	网页里建任务 → 云上完成	托管、有浏览器、数据出本机

差异点在哪：同样一句话，**dsh 让你多了一个选择维度——界面和工具链都可以换**。其他 Agent 的界面/工具链是官方定的，dsh 是你拼的。

案例对比：真实工作流（我们的实测）

以下是我们**真实开发 dsh 生态**时的对比观察（2026-08-13）：

场景	dsh 实测	备注
简单文件创建	冷启动 ~110s（首轮含上下文注入）→ 热缓存 ~1s	思考档位是主要变量
50 步工具链任务	LLM 耗时 10m+，工具调用 9m+	每步思考累计—— 提速插件价值在此
插件开发	从零到可运行插件：1 天（含测试+实机验证）	扩展点清晰（agent/request waterfall）
与 Claude Code 同任务	dsh 配 V4-Flash 成本约为 Claude 的 1/10~1/30	价格维度 dsh 生态显著占优

同模型 × 不同 Agent 的严格对比见 [Benchmark 附录](./benchmark.md)（omp 36s / dsh 85s / opencode 114s）。

1.5 什么时候用 dsh（选型决策）

 **推荐入场：**

 **暂时观望：**

1.6 常见问题（FAQ）

Q1：dsh 是模型吗？ 不是。dsh 是运行时/框架，模型通过 llm 插件接入（官方适配 DeepSeek V4 系，理论上可接其他 OpenAI 兼容模型）。

Q2: dsh 和 OpenCode 什么关系? 都是开源 Agent 客户端, 但定位不同: OpenCode 是"客户端 + 配置", dsh 是"运行时 + 官方插件生态" (有官方后端 bundle 与 60+ 官方包)。dsh 更底层、可定制面更大。

Q3: 没写过 TypeScript 能玩吗? 能。使用 (第 2 章) 不需要编程; 写插件 (第 4 章) 需要基础 TS, 但教程给完整代码。

Q4: dsh 稳定吗? 当前 rc 阶段 (0.1.0-rc.6), 迭代快、有破坏性变更。生产核心依赖建议等正式版; 玩生态现在正是时机。

Q5: 为什么现在学 dsh 值得? 生态零日 + 官方点名鼓励社区 + 中文教程空白——**每个早期生态都有"第一个吃螃蟹的人"的红利**, 现在是入场窗口。

下一章: [第 2 章: 五分钟快速上手](./02-quickstart.md) —— 装起来, 跑起来。

动手练习 (检验你是否真懂了)

完成练习后, 进 [第 2 章](./02-quickstart.md) 动手装起来。

第 2 章：五分钟快速上手

本章目标：**跟着做，跑起来**。每一条命令都给出预期输出与常见错误解法。建议打开终端边看边做。

TL;DR（本章核心，30 秒版）

2.1 准备工作（30 秒检查）

需要	检查命令	通过标准
Node.js ≥ 22	<code>node --version</code>	v22.x 或更高（推荐 24）
npm（随 Node 附带）	<code>npm --version</code>	有版本号即可
网络	能访问 npm registry	能装包
（可选）DeepSeek API Key	<code>https://platform.deepseek.com</code>	用于真实对话

没有 API Key 也能启动 dsh（界面能开），但对话需要 Key。本白皮书示例假设已配置。

2.2 安装（两种方式）

方式一：直接运行（推荐新手）

```
npx -y @deepseek-ai/dsh --version
```

首次运行会下载 dsh（包体较大，含 40+ 插件模块，约 1-3 分钟）。看到版本号即成功：

```
0.1.0-rc.6
```

方式二：全局安装（推荐频繁使用）

```
npm install -g @deepseek-ai/dsh
dsh --version
```

2.3 模式一：Web UI (dsh web)

启动

```
dsh web
```

预期输出：

```
dsh web: http://127.0.0.1:3080
```

浏览器打开 <http://127.0.0.1:3080>。

界面认识（对照截图）

[图: dsh Web UI 对话]

区域	内容
左栏	会话列表 / 工作区切换 / 新建会话
中栏	对话区：输入框、模型选择（DeepSeek V4 Flash）、推理等级（High）
右侧/底部	插件侧边栏（默认空；安装社区插件后出现）
右上	Session log（会话日志） / 轨迹（工具调用轨迹）

第一次对话

预期回复类似：

你好！我是 DeepSeek 驱动的 AI 编程助手，可以帮你写代码、调试问题、处理文件、搜索资料，以及完成各种开发和办公任务。

模型与推理档位

点输入框旁的「选择模型」，打开模型与推理档位选择器：

[图: dsh 模型选择与推理档位]

模型	定位
deepseek-v4-flash (默认)	性价比：快、便宜，日常够用
deepseek-v4-pro	旗舰：更强，更贵更慢

推理等级（思考模式三档，2026-08-13 起支持）：

档位	速度	质量	建议场景
low	最快	够用	简单/确定性任务、批量、工具链廉价轮次
high (默认)	中等	好	日常 Agent 任务
max	最慢	最强	复杂推理、长链规划

💡 **性能关键认知：**模型在**每次工具调用前都会重新思考**。实测一个"创建文件"任务，思考占~90% 墙钟时间；50 步工具链任务思考累计可达数分钟到十几分钟。**调低推理档位是性价比最高的提速手段**（见第 6 章 + dsh-tool-turbo 插件）。

2.4 模式二：Headless（一次性任务，适合脚本/CI）

```
dsh --profile headless "你好，请用一句话介绍你自己"
```

预期输出（打印结果后进程退出）：

```
你好！我是 DeepSeek 驱动的 AI 编程助手，可以帮你写代码、调试问题、处理文件、搜索资料，以及完成各种开发和办公任务。
```

Headless 的核心价值：

实战：写个脚本每天跑一次"生成日报"：

```
dsh --profile headless "读取工作区今天的 git log，生成一份中文日报摘要" > daily-report.md
echo "exit=$?"
```

2.5 你的第一个插件：给 web 加个 Git 面板

dsh 的侧边栏默认是空的——安装社区插件 `dsh-better-sidebar` 体验"一切皆插件"（详细原理见第 3 章，这里先跑通）：

```
# 1. 找到你的 web profile
#   Windows: %USERPROFILE%\dsh\profiles\web
#   macOS/Linux: ~/.dsh/profiles/web

# 2. 在 package.json 的 dependencies 加一行（link: 指向插件源码）
#   "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"

# 3. 在 cordis.patch.yml 加挂载行
#   - insert:
#       - id: better-sidebar
#       name: dsh-better-sidebar

# 4. 安装并重启
cd ~/.dsh/profiles/web && pnpm install
dsh web
```

重启后，右侧出现文件管理 / 终端 / **Git 面板** / 浏览器等标签：

[图: dsh Git 面板 (better-sidebar 插件)]

图中「拉取远端 / 拉取合并 / 推送」按钮是社区 PR 实现的（见第 5 章案例）——**这就是"插件生态"的运转方式。**

2.6 配置与目录速查

首次运行后生成的目录：

```
~/dsh/
├─ settings.yaml      # 全局设置（模型、推理档位）
├─ profiles/          # profile 目录
│   └─ web/
│       ├── package.json    # 插件依赖 + 清单
│       └─ cordis.patch.yml # 补丁层（挂载插件）
├─ sessions/          # 会话数据
└─ storages/           # 持久化存储
```

settings.yaml 示例:

```
agent-default-model:
  model: deepseek-v4-flash
  reasoningEffort: high
```

2.7 命令速查

命令	用途
dsh web	启动 Web UI (=dsh --profile web)
dsh --profile headless "任务"	一次性任务，打印结果退出
dsh plugin --profile <name> add <pkg>	给 profile 安装插件
dsh --dump-config	打印合成配置树
dsh --profile tui	TUI 模式（需先安装 tui 插件，官方未内置）
dsh --version	版本

2.8 排障速查

现象	原因与解法
dsh: profile "tui" does not exist	tui profile 需插件创建 (dsh plugin --profile tui add <pkg>)
npm 极慢	首次下载包体大; npm i -g 后更快
浏览器打不开 3080	端口被占: netstat -ano findstr 3080 → kill PID
模型无响应	检查 ~/.dsh/settings.yaml 模型配置 + API Key

插件装不上 (404)	rc.1 依赖断裂 : 确认依赖用 <code>^0.1.0-rc.6</code> 线 (第 3 章常见坑 #1)
升级后行为变了	rc 阶段破坏性变更正常, 看官方 changelog

下一章: [第 3 章: profile 与插件系统](./03-profiles.md) —— 理解可定制骨架。

动手练习 (10 分钟内完成)

全部通过后, 进 [第 3 章](./03-profiles.md) 理解"为什么能这样改"。

第 3 章：profile 与插件系统

本章目标：理解 dsh 的可定制骨架——profile 怎么组织、插件怎么挂载、host/client 双半是什么。这是从“用户”变成“开发者”的分水岭。

TL;DR（本章核心，30 秒版）

3.1 profile：一个可启动的配置栈

dsh 用 **profile** 表示“一种可启动的形态”。官方内置两个，其余用插件创建：

profile	用途	命令
web	Web UI（对话 + 侧边栏 + 工具）	dsh web
headless	一次性 CLI 任务	dsh --profile headless "任务"
tui（需插件）	终端 UI	dsh --profile tui（未内置，需安装插件）

一个 profile 目录长这样（`~/.dsh/profiles/<name>/`）：

```
profiles/web/
├─ package.json      # 插件依赖 + dsh.profile 清单（bundles 顺序）
├─ cordis.patch.yml  # 你的补丁层：挂载/覆盖插件的声明
├─ cordis.yml        # （生成的）合成配置
├─ pnpm-workspace.yml
└─ node_modules/
```

加载顺序（官方文档原文）：内置 bundle（dsh-base → dsh-web-app）→ profile 的 cordis.patch.yml → 用户级 `~/.dsh/cordis.patch.yml` → `--patch` 覆盖层。

3.2 挂载一个插件：两处改动

以挂载社区插件 dsh-better-sidebar 为例（真实操作）：

- ① ``package.json`` 加依赖（link：指向本地源码，或用 npm 包名）：

```
{
  "dependencies": {
    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"
  }
}
```

② `cordis.patch.yml` 加挂载行:

```
- insert:
  - id: better-sidebar
    name: dsh-better-sidebar
```

③ 安装并重启:

```
cd ~/.dsh/profiles/web && pnpm install
dsh web # 重启后生效
```

💡 插件默认是**按会话隔离**的（better-sidebar 的布局/标签按会话持久化）——挂载是 profile 级，但状态是会话级。

3.3 host 半与 client 半：一个包，两副面孔

插件可以同时携带两个运行半：

半边	运行位置	职责	示例
host 半	Node 进程	工具、服务、事件、文件系统、进程	apply(ctx) 注册工具/服务
client 半	浏览器 (web profile)	UI、交互、DOM	package.json 的 dsh.client 声明 + src/client/

package.json 里如何声明 client 半:

```
{
  "dsh": {
```

```
"client": {
  "inject": ["@deepseek-ai/dsh-client-runtime", "@deepseek-ai/dsh-client-locale"],
  "platform": "web"
},
"exports": {
  ".": { "types": "./lib/types/index.d.ts", "default": "./lib/index.js" },
  "./client": { "types": "./lib/types/client/index.d.ts", "default":
"./lib/client.js" }
}
```

3.4 扩展点：改行为优先找钩子，别 fork 核心

官方原则（CONTRIBUTING/AGENTS.md 明示）：**"Plugins, not loop changes: new behavior goes on documented extension points"**。新手最常见的错误是改核心 loop——正确做法是用扩展点。

已确认的常用扩展点（后续章节逐一实战）：

扩展点	位置	用途
agent/request waterfall	agent-loop	每次模型请求前改配置（provider/model/reasoningEffort/tools）——tool-turbo 用这个
agent/request-error	agent-loop	请求失败时干预（官方 compaction 插件用这个做上下文溢出恢复）
conversationEvents.register	client runtime	订阅/注入对话事件（tool/call、turn/start 等）
ctx.slots.inject	client ui-slots	在界面槽位注入 UI（如 turnTail 显示产物文件行）
settings 服务	dsh-settings	注册用户可配置的命名空间（设置页自动渲染）
ctx.provide / ctx.get	cordis	跨插件提供服务

3.5 常见坑（真实踩过）

坑	现象	解决
rc.1 依赖断裂	pnpm install 报 @deepseek-ai/dsh-type-meta@0.0.1-rc.1 404	官方 rc.1 时代多个包从未发布；升级到 ^0.1.0-rc.6 线
插件缺 main	dsh: No "exports" main defined	host 插件 package.json 要暴露 . 入口 ("main": "src/index.ts" 可被 tsx 直接加载)
事件 handler 忘了 await next()	请求丢失 provider/model 报错	agent/request 的 next() 返回 Promise ，必须 await 后 spread
类型报 'agent/request' is not assignable to keyof Events	npm 类型未 re-export 官方类型增强	用宽松签名 (ctx.on as unknown as ...) 在边界转换
client 包产物依赖 window.__ModuleLoader__	jsdom 无法直接跑 client 测试	组件级测试需 dsh web 运行时 (官方 CI 跑)

动手练习（检验你是否真懂了）

- 自查：参考本章 3.1 节"profile 目录长这样"
- 自查：参考本章 3.3 节表格
- 自查：参考本章 3.1 节"加载顺序"段落
- 自查：参考本章 3.2 节的两处改动示例
- 自查：对比本章 3.5 节"常见坑"表格
- 自查：参考本章 3.4 节扩展点表格 + "改行为优先找钩子"原则

常见疑问 FAQ

Q1：profile 和 bundle 有什么区别？ profile 是"一种可启动形态"（如 web、headless）， bundle 是"一组预配置的插件集合"。profile 通过 dsh.profile.bundles 指

定要加载哪些 bundle，再叠加自己的 patch。简单说：bundle 是积木包，profile 是拼好的成品。

Q2: `cordis.patch.yml` 里的 `insert` 是什么意思？还有其他操作吗？ insert 是"在插件链中插入一个新插件"。patch 文件基于 cordis 的配置合并机制，支持 insert（插入）、override（覆盖已有插件配置）等操作。新手只需掌握 insert 即可挂载大部分插件。

Q3: 插件状态是按 profile 隔离还是按会话隔离？ 挂载是 profile 级的（装一次，该 profile 下所有会话都能用），但状态是会话级的。比如 better-sidebar 的标签布局按会话持久化，不同会话互不干扰。

Q4: 我想写一个只有 client 半的插件（纯 UI），可以吗？ 可以。package.json 里声明 dsh.client + exports["./client"], 不写 exports["."] 即可。但注意：client 半无法直接访问文件系统或跑命令，需要通过 host 半的服务（ctx.provide/ctx.get）桥接。

Q5: `agent/request` waterfall 和 `conversationEvents.register` 有什么区别？我该用哪个？ agent/request 是"改模型请求配置"的钩子

（provider/model/tools/reasoningEffort），每次请求前触发，返回新配置。

conversationEvents.register 是"订阅/注入对话事件"的钩子（tool/call、turn/start 等），用于监听或注入事件流。想改请求参数用前者，想监听对话流用后者。

Q6: 为什么我的插件装上去没反应？怎么调试？ 三步排查：① 确认 pnpm install 无报错且 node_modules 里有你的插件；② 确认 cordis.patch.yml 的 id/name 拼写正确；③ 重启 dsh web 后看进程日志有没有插件加载信息。如果 host 半有 console.log，日志里应该能看到输出。

下一章：[第 4 章：插件开发实战](./04-plugin-dev.md)（规划中）—— 从零写第一个 host 插件。

第 4 章：插件开发实战

本章目标：从零写一个**真实可用的 host 插件**——通过 `agent/request` 扩展点自动调节推理档位。这是社区项目 `dsh-tool-turbo` 的完整拆解，所有代码可运行、可测试。

TL;DR (本章核心, 30 秒版)

4.1 我们要做什么

问题：`dsh` 在每次工具调用前模型都会重新思考 (`reasoning_effort`)。一个 50 步工具链任务，“思考”占 90%+ 墙钟时间。

方案：一个 `host` 插件，监听 `agent/request waterfall`，根据当前步骤最近的工具调用，把简单轮次的 `reasoning_effort` 从 `high` 降到 `low`。

4.2 项目骨架

```
dsh-tool-turbo/
├─ package.json          # host 插件声明
├─ tsconfig.json
├─ src/
│  ├─ effort-decision.ts  # 纯函数：决策逻辑（零依赖，可单测）
│  └─ index.ts            # apply(ctx): 接入扩展点
└─ tests/
   └─ effort-decision.spec.ts
```

`package.json` 关键字段：

```
{
  "name": "dsh-tool-turbo",
  "type": "module",
  "main": "src/index.ts",
  "exports": {
    ".": { "types": "./src/index.ts", "default": "./src/index.ts" }
  },
  "peerDependencies": {
    "@deepseek-ai/cordis": "^4.0.1",
    "@deepseek-ai/dsh-agent": "^0.1.0-rc.6"
  }
}
```

```
}
```

△ 依赖版本务必用 `^0.1.0-rc.6` 线——`rc.1` 线的 npm 依赖链是断的（见第 3 章常见坑）。

4.3 纯函数：决策逻辑（零依赖，可单测）

`src/effort-decision.ts`:

```
export type EffortId = 'low' | 'high' | 'max'

export interface ToolCallSample {
  name: string      // 工具名, 如 'write'、'read'、'bash'
  argsSize: number  // 参数大小 (字符数)
}

export interface EffortDecisionInput {
  recentCalls: readonly ToolCallSample[]
  selected: EffortId      // 用户基线档
  allowDowngrade: boolean
  allowUpgrade: boolean
}

const SIMPLE_TOOL_RE = /^(fs|bash|terminal|read|write|grep|glob|edit|ls|cat|rm|cp|touch|mkdir|pwd)/i
const HEAVY_ARGS = 800

export function decideEffort(input: EffortDecisionInput): EffortId {
  const { recentCalls, selected, allowDowngrade, allowUpgrade } = input
  if (recentCalls.length === 0) return selected    // 全新提示: 保持基线

  const ratio = recentCalls.filter(c =>
    SIMPLE_TOOL_RE.test(c.name) && c.argsSize < HEAVY_ARGS,
  ).length / recentCalls.length
  const heaviest = recentCalls.reduce((m, c) => Math.max(m, c.argsSize), 0)

  if (ratio >= 0.75 && allowDowngrade) return 'low'
  if (heaviest >= HEAVY_ARGS * 4 && allowUpgrade) return 'max'
  if (ratio < 0.75) return allowUpgrade ? 'high' : selected
  return selected
}
```

为什么拆成纯函数：决策逻辑与 dsh 运行时解耦——单元测试零依赖、毫秒级、覆盖所有分支，实机只需要验证“注入是否真的发生”。

4.4 插件主体：接入 agent/request waterfall

src/index.ts:

```
import type { Context } from '@deepseek-ai/cordis'
import { decideEffort, type ToolCallSample } from './effort-decision.ts'

export interface ToolTurboConfig {
  enabled: boolean
  allowDowngrade: boolean
  allowUpgrade: boolean
  baseline: 'low' | 'high' | 'max'
}

export const DEFAULT_CONFIG: ToolTurboConfig = {
  enabled: true, allowDowngrade: true, allowUpgrade: false, baseline: 'high',
}

const WINDOW = 8

function recentToolCalls(agent: unknown): ToolCallSample[] {
  const events = (agent as { session?: { events?: readonly unknown[] } }).session?.events ?? []
  const out: ToolCallSample[] = []
  for (let i = events.length - 1; i >= 0 && out.length < WINDOW; i--) {
    const e = events[i] as { type?: string; data?: { name?: string; arguments?: unknown } } | undefined
    if (e?.type !== 'tool/call') continue
    out.push({
      name: e.data?.name ?? 'tool',
      argsSize: typeof e.data?.arguments === 'string' ? e.data.arguments.length : 0,
    })
  }
  return out.reverse()
}

export function apply(ctx: Context, config: ToolTurboConfig = DEFAULT_CONFIG): void {
  if (!config.enabled) return

  // 边界适配：npm 包未 re-export 官方事件类型增强，这里放宽签名（第 3 章常见坑）
```

```
const on = ctx.on as unknown as (
  event: string,
  handler: (payload: Record<string, unknown>, next: () => unknown) => unknown |
  Promise<unknown>,
) => void

on('agent/request', async (payload, next) => {
  const seed = await next() as { reasoningEffort?: unknown } // ⚠ 必须 await!
  const calls = recentToolCalls(payload.agent)
  const effort = decideEffort({
    recentCalls: calls,
    selected: config.baseline,
    allowDowngrade: config.allowDowngrade,
    allowUpgrade: config.allowUpgrade,
  })
  console.log(`[tool-turbo] calls=${JSON.stringify(calls)} => reasoningEffort=${
    {effort}}`)
  return { ...seed, reasoningEffort: effort }
})
}
```

三个关键点（都是真实踩过的坑）：

4.5 测试

单元测试（纯函数，零依赖）：

```
import { describe, expect, it } from 'vitest'
import { decideEffort } from '../src/effort-decision.ts'

it('downgrades to low for simple tool chains', () => {
  expect(decideEffort({
    recentCalls: [{ name: 'write', argsSize: 40 }],
    selected: 'high', allowDowngrade: true, allowUpgrade: true,
  })).toBe('low')
})
// ... 更多分支：全新提示保持基线 / 禁用降档 / 超大载荷升 max / 混合工具升 high
```

实机验证（关键——证明“注入真的发生”）：

挂载插件（第 3 章方法）→ 重启 dsh web → 发一个创建文件的任务 → 观察 dsh 进程日志：

```
[tool-turbo] agent/request: calls=[] => reasoningEffort=high
```

```
[tool-turbo] agent/request: calls=[{"name":"write",...}] => reasoningEffort=low
```

第一轮无工具调用 → 保持基线 high；检测到 write 工具 → 下一轮降为 low。**注入链路完整工作。**

完整可运行代码：<https://github.com/Electricitysheep/dsh-tool-turbo>

4.6 给新手的三条开发纪律

动手练习（检验你是否真懂了）

自查：参考本章 4.2-4.5 节的文件结构

自查：参考本章 4.3 节“为什么拆成纯函数”段落

自查：参考本章 4.5 节单元测试示例，预期结果取决于 allowUpgrade 配置

自查：参考本章 4.4 节“三个关键点”第 1 条

自查：参考本章 4.3 节纯函数设计模式，关键是输入输出类型定义

自查：参考本章 4.4 节“waterfall 语义”段落 + 第 3 章 3.4 节扩展点说明

常见疑问 FAQ

Q1: `next()` 返回的 seed 里到底包含什么字段？ seed 是当前 waterfall 链上游累积的请求配置，通常包含 provider、model、reasoningEffort、tools 等字段。你的监听者拿到 seed 后，spread 覆盖想改的字段，其余原样传递。具体字段以官方 agent-loop 包的类型定义为准（rc 阶段可能变动）。

Q2: 为什么 npm 包的类型签名要“放宽”？不能直接用官方类型吗？ 因为 npm 发布的 @deepseek-ai/dsh-agent 等包没有 re-export 内部的事件类型增强（Events 接口没有被 module augmentation 扩展）。直接用 ctx.on('agent/request', ...) 会报类型错误。解决方案是在边界用 as unknown as 转换，这是 rc 阶段的临时方案，正式版可能修复。

Q3: 我的插件需要在每个工具调用后都触发决策, `agent/request` 够吗? 够。

`agent/request` 在每次模型请求前触发 (包括工具调用后的下一轮请求)。你只需要在监听者里读取最近的工具调用历史 (从 `payload.agent.session.events` 里倒序找 `tool/call` 事件), 就能做按步决策。

Q4: 纯函数测试通过了, 实机验证也通过了, 但用户反馈"有时候没效果", 可能是什么原因?

几种可能: ① 用户的 `settings.yaml` 里 `reasoningEffort` 是 `low`, 降档无效果 (已经最低了); ② 用户的任务全是简单工具, 本来就快, 感知不到差异; ③ `waterfall` 里有其他插件覆盖了你的返回值。建议加日志记录每步的输入/输出档位。

Q5: 我想给插件加一个用户可配置的开关 (设置页里能开关), 怎么做? 用 `settings` 服务注册一个命名空间 (如 `tool-turbo`), 声明配置项 (`enabled`、`baseline` 等)。dsh 的设置页会自动渲染表单, 用户修改后通过 `ctx.get` 读取。具体 API 参考官方 `dsh-settings` 包文档。

Q6: `recentToolCalls` 函数里为什么要 `reverse()`? 因为从 `events` 数组末尾往前遍历 (取最近的 `N` 个), 结果是倒序的 (最新的在前)。reverse 后恢复时间正序 (最旧的在前), 和实际调用顺序一致, 方便决策逻辑按"最近窗口"理解。

下一章: [第 5 章: 实战案例](./05-cases.md) (规划中) —— Git 面板、HTML 草稿预览、提速插件。

第 5 章：dsh 能帮你做什么——应用场景全景

本章目标：抛开"如何开发 dsh"的视角，站在**使用者**角度，看 dsh 能在哪些日常场景真正帮到你。每个场景都有可复制的任务示例与真实耗时参考。

TL;DR（本章核心，30 秒版）

5.1 场景总览

场景	典型任务	dsh 的工具	实测参考
日常编码	写函数、修 bug、补测试	write / read / bash / grep	5-bug 修复+49 测试：94s
数据处理	清洗、分析、可视化、报告	read / write / bash / python	数据清洗+可视化：186s
文档知识	API 文档、教程、总结、翻译	read / write	API 文档生成：1m04s
自动化运维	批量处理、CI 任务、日志分析	bash / headless	headless 脚本化
代码理解	重构、审查、技术调研	grep / glob / read / bash	代码重构：4m37s

所有实测来自白皮书第 10 章真实案例（合成数据），详见 [benchmark](./benchmark.md)。

5.1.1 高缓存命中率：dsh 的成本秘密武器

这是 dsh 生态最被低估的优势。DeepSeek API 的 **context cache**（提示词缓存）：重复/相似的输入 token 按缓存价计费。实测（真实 dsh 会话统计）**缓存命中率可达 97%**。

为什么 dsh 缓存命中率特别高：

原因	说明
会话延续	同会话多轮对话，系统提示/技能目录/历史重复 → 命中

稳定的工具 schema	每步请求携带相同的工具定义 → 命中
Agent 工作负载特性	多步工具链反复携带相同上下文 → 命中率天然高

成本影响（98% 缓存折扣）：

计费项	未命中价	缓存命中价	折扣
输入（Flash）	\$0.14/M	\$0.0028/M	98%
输入（Pro）	\$0.435/M	\$0.003625/M	99%+

实际意义：一个长会话 Agent 任务，绝大部分输入 token 走缓存价——**实测中"50 步工具链任务、2.4M 输入 token"的成本远低于按未命中价计算的预期。**对高频/批量/长会话场景，这是数量级的成本优势。

怎么让缓存命中率更高：**保持会话延续**（不要频繁新开会话）、**稳定 prompt 前缀**（系统提示/技能目录不要反复变化）、批量任务放同一会话/同前缀。

5.2 场景一：日常编码（最适合入门的场景）

典型任务：

任务示例（headless 一键跑）：

```
dsh --profile headless "审查 buggy_calculator.py：找出所有 bug 修复，写完整测试，运行验证全部通过"
# 实测：5 个 bug 全修复 + 49 个测试通过（94s）
```

为什么 dsh 适合：编码是工具链最完整的场景——读文件、写代码、跑命令、看结果，闭环顺畅。产物（改的文件）会在对话末尾直接可打开。

提示词技巧：一定要写"运行验证"——dsh 会照着闭环；只写"帮我修 bug"它会修完就停。

5.3 场景二：数据处理与分析

典型任务：

任务示例：

```
dsh --profile headless "分析 sales_data.json 的数据质量（缺失/类型错/重复/异常），写清洗脚本+可视化脚本，运行验证，总结处理方案"  
# 实测：52→35 行归零 + 图表 + 权衡说明（186s）
```

亮点（实测）： dsh 不仅执行，还会**说明权衡**——"中位数填充会产生尖峰""异常值是否该删除取决于业务"——这是 agent 工程层"会思考"的直接证据。

5.4 场景三：文档与知识工作

典型任务：

任务示例：

```
dsh --profile headless "为 user_module.py（8 个函数）生成完整 Markdown API 文档：参数表、返回值、异常、示例、边界情况"  
# 实测：423 行 ReadTheDocs 风格文档（1m04s），主动识别"文档契约 vs 代码实现"差异
```

亮点（实测）： dsh 会**防御式写作**——标注调用陷阱（负数 limit 行为、空参数行为）、发现文档声明与实现不符的地方如实说明。

5.5 场景四：自动化与运维（headless 是王牌）

典型任务：

任务示例（脚本化）：

```
# cron 每天 8 点生成日报  
dsh --profile headless "读取工作区 git log，生成中文日报" > daily-report.md  
  
# 批处理：目录里所有 .txt 转 .md  
dsh --profile headless "把 docs/ 下所有 .txt 转成 .md（保留结构）"
```

为什么 headless 是王牌：进程级工具（bash/文件/搜索）全可用 + 非零退出码即失败 + 输出可管道——它是“能写进 CI 的 Agent”。

5.6 场景五：代码理解与重构

典型任务：

任务示例：

```
dsh --profile headless "把 legacy_orders.py 重构为面向对象+职责分离，保持行为一致，写测试验证重构前后输出相同"  
# 实测：17/17 测试通过，含脚本级产物对比（4m37s）
```

亮点（实测）：dsh 的工程意识——识别重复代码合并单一入口、保持向后兼容、**发现沙箱下 tempfile 权限问题主动换方案。**

5.7 让场景落地的三个提示词法则

5.8 我的第一个真实任务（15 分钟上手）

动手练习

常见疑问（FAQ）

下一章：[第 6 章：进阶与性能调优](./06-advanced.md)

5.9 行业视角：dsh 在不同领域的打开方式

同一套工具链，不同行业的用法。以下为通用场景（不含真实业务数据），合规敏感领域（医疗/法律）请务必人工审核。

金融与投研

教育与学习

销售与客服

运营与内容

研究与综述（学术/技术）

⚠ 合规提醒

医疗诊断、法律意见、财务建议等**高风险场景**：dsh 输出需人工审核，不能直接用于决策——
—工具执行有沙箱，但**内容责任在使用者**。

第 6 章：进阶与性能调优

本章目标：从"能跑"到"跑得好"——推理档位策略、工具调用耗时分析、真实踩坑清单。

TL;DR（本章核心，30 秒版）

6.1 性能模型：dsh 的时间花在哪

实测一个"创建文件"任务的耗时分布：

阶段	占比	说明
模型思考（Think）	~90%	每次工具调用前都会重新思考，是绝对大头
工具执行	<1%	文件写入等毫秒级
网络/渲染	~10%	API 往返 + UI 更新

推论：

6.2 reasoning_effort 策略（官方三档）

档位	场景建议
low	简单/确定性轮次：文件操作、批量、工具链中的廉价步
high	日常 Agent 任务（默认）
max	复杂推理、长链规划、debug

手动： UI 的"推理等级"选择器，或 `~/.dsh/settings.yaml` 的 `reasoningEffort`。**自动：** 插件按工具轮次动态降档（`dsh-tool-turbo`，第 4 章）。

6.3 工具调用耗时可视化

dsh 的会话统计行（Web UI 底部）显示：`N 轮 · M 步 | LLM Xs · 工具调用 Ys | 首 token 平均 ...`——这是最快的瓶颈定位手段。

进阶：host 插件监听工具事件做 per-tool 计时（tool-turbo 已内置 host 日志版本），把"哪个工具最慢"暴露出来。

6.4 常见坑清单（真实踩过，含解法）

#	坑	现象	解法
1	rc.1 依赖断裂	pnpm install 404 (dsh-type-meta 等从未发布)	依赖用 ^0.1.0-rc.6 线
2	插件缺 main	No "exports" main defined	暴露 . 入口; "main": "src/index.ts" 可被 tsx 加载
3	next() 忘 await	provider/model 丢失报错	agent/request 的 next() 返回 Promise, 必须 await
4	事件类型不识别	'agent/request' is not assignable to keyof Events	npm 未 re-export 类型增强, 边界放宽签名
5	client 测试跑不了	jsdom 报 window.__ModuleLoader__ undefined	client 产物依赖 dsh 引导机制, 组件测试在官方 CI 跑
6	简单任务"突然变快"的误判	1s vs 110s 差异被误归因	DeepSeek context cache 命中也会提速——A/B 测试要用全新 prompt
7	端口占用	dsh web 起不来	`netstat -ano

6.5 评测视角：官方成绩单 vs 独立实测

（结合 0813 正式版发布）看 Agent 模型成绩单要三问：

dsh 的 agent/request waterfall 让模型跑分可复现——这是它相比闭源产品的工程优势。

动手练习（检验你是否真懂了）

自查：参考本章 6.1 节耗时分布表格

自查：参考本章 6.2 节三档策略表格

自查：参考本章 6.3 节"耗时可视化"段落

自查：参考本章 6.2 节"手动"段落

自查：参考本章 6.4 节坑 #7

自查：参考本章 6.5 节"官方 Terminal-Bench 87.9 vs AA 独立 79"的例子

常见疑问 FAQ

Q1：为什么我的 dsh "有时候特别快，有时候特别慢"？ 两个主要原因：① DeepSeek context cache 命中时，首轮上下文注入会快很多（1s vs 110s 的差异可能来自这里）；② 推理档位不同，思考时间差几倍。做 A/B 对比测试时，要用全新 prompt（避免缓存命中）+ 固定档位。

Q2：`dsh-tool-turbo` 真的能提速多少？有没有实测数据？ 提速收益取决于任务类型。简单任务（1-3 步）dsh 本来就快，插件效果不明显。**长工具链任务（20-50 步）收益最大**，因为每步思考降档的累计效果显著。具体数据参考 tool-turbo 仓库的实测日志。

Q3：我把推理档位设为 `low`，模型质量会下降很多吗？ 取决于任务。简单文件操作、批量处理、确定性任务用 low 质量几乎无差别。复杂推理、长链规划、debug 用 low 可能漏掉关键步骤。建议：日常用 high，批量/简单任务手动切 low 或装 tool-turbo 自动降档。

Q4：坑 #6 说的"context cache 命中"是什么？怎么判断是否命中？ DeepSeek API 会对重复/相似的上下文做缓存，命中时首 token 时间大幅缩短。判断方法：看 dsh 会话日志里的"首 token 平均"指标，如果某次突然从 10s+ 降到 1s 以下，大概率命中了缓存。做性能对比测试时，用全新 prompt 避免缓存干扰。

Q5: rc 阶段的破坏性变更会影响性能吗？升级后需要注意什么？ 可能影响。rc 阶段的 agent-loop、waterfall 签名、插件 API 都可能变。升级后：① 跑一遍 `dsh --version` 确认版本；② 看官方 changelog 有没有 breaking changes；③ 跑一个你熟悉的任务对比耗时和行为；④ 自定义插件检查类型兼容性。

Q6: 我想做性能基准测试 (benchmark)，有什么建议？ 三个原则：① 固定环境（同模型、同网络、同档位）；② 用全新 prompt 避免缓存；③ 多次运行取中位数（单次波动大）。记录：墙钟时间、LLM 耗时、工具调用耗时、首 token 时间。参考本章 6.1 节的耗时分布模型设计你的 benchmark。

下一章：[第 7 章：生态与资源](./07-ecosystem.md)（规划中）。

6.6 缓存策略：让每次调用都更便宜

第 5 章讲了高缓存命中率的价值（实测 97%、98% 折扣），这里给出**可操作策略**：

目标	做法
提高命中	长任务 保持会话延续 （避免频繁新建会话）
提高命中	系统提示/技能目录等 前缀保持稳定 （不要频繁改配置）
提高命中	批量任务放同一会话/同前缀（如同一批文件分析）
降低成本	简单轮次用 low 档（思考 token 减少 → 总 token 下降）
监控	会话统计行看“缓存命中 %”（Web UI 底部）——低于预期就检查前缀稳定性

成本模型速记：总成本 ≈ 输出 token×输出价 + 输入未命中×未命中价 + 输入命中×命中价
——Agent 工作负载输入占比高，**缓存命中率是成本的第一变量**。

第 7 章：生态与资源

本章目标：给你一份"加入 dsh 生态"的地图——官方入口、社区现状、插件实操、以及参与方式。

TL;DR (本章核心, 30 秒版)

7.1 官方入口

资源	地址	用途
官方仓库	https://github.com/deepseek-ai/deepseek-harness	源码、架构文档、issue
API 文档	https://api-docs.deepseek.com	模型、定价、API 指南
Discord	官方 README 内链接	社区讨论
Discussions	官方仓库 Discussions	提案/求助（官方当前建议的贡献入口）

注意：官方 CONTRIBUTING 明确"目前不接受外部 PR"（2026-08-13 时点）——但鼓励：

7.2 插件生态 (2026-08-13 快照)

项目	定位	状态
DSH-better-sidebar	文件管理/终端/Git/浏览器侧边栏	社区最完整插件
dsh-tool-turbo	工具调用提速 (reasoning_effort 自动调节)	社区提速插件
dsh-handbook (本白皮书)	新手教程	生态文档

发现插件：GitHub 搜 `topic:dsh-plugin`。 **发布插件：**给你的仓库加 `dsh-plugin` topic + npm 发布。

7.3 如何参与生态（新手路径）

7.4 推荐阅读路径

目标	路径
快速上手	第 2 章 → 装 better-sidebar → 日常用
开发插件	第 3 章 → 第 4 章 → 抄第 5 章案例
性能调优	第 6 章 → tool-turbo 源码
深度定制	官方 AGENTS.md（架构） → docs/architecture.md → packages/ 源码

7.5 真实插件安装实操

以下给出两个代表性插件的**完整挂载步骤**（语法与第 3 章 3.2 节一致）。

安装 DSH-better-sidebar

① ``package.json`` 加依赖（`~/ .dsh/profiles/web/package.json`）：

```
{
  "dependencies": {
    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"
  }
}
```

② ``cordis.patch.yml`` 加挂载行：

```
- insert:
  - id: better-sidebar
    name: dsh-better-sidebar
```

③ 安装并验证：

```
cd ~/ .dsh/profiles/web && pnpm install && dsh web
```

重启后观察左侧是否出现文件管理/终端/Git/浏览器四个标签页。

安装 dsh-tool-turbo

① `package.json` 加依赖：

```
{
  "dependencies": {
    "dsh-tool-turbo": "link:C:\\path\\to\\dsh-tool-turbo"
  }
}
```

② `cordis.patch.yml` 加挂载行：

```
- insert:
  - id: tool-turbo
    name: dsh-tool-turbo
```

③ 安装并验证：

```
cd ~/.dsh/profiles/web && pnpm install && dsh web
```

发一个“创建 3 个文件”的任务，观察日志是否出现 `[tool-turbo] calls=[...] =>`

`reasoningEffort=low`（参考第 4 章 4.5 节）。

⚠ 两个插件均要求 `@deepseek-ai/dsh-agent ^0.1.0-rc.6`，若 `pnpm install` 报 404，请检查版本线（见第 3 章 3.5 节）。

7.6 发现插件的方法

渠道	具体操作	预期结果
GitHub topic	https://github.com/topics/dsh-plugin 或搜索 topic:dsh-plugin	所有打了 topic 的仓库

GitHub 全局搜索	dsh-plugin + 语言过滤 TypeScript	含关键词的仓库与代码
npm 搜索	npm search dsh-plugin 或 https://www.npmjs.com/search?q=dsh-plugin	已发布的包（rc 阶段以 GitHub 为主）
官方讨论区 Show and tell	https://github.com/deepseek-ai/deepseek-harness/discussions/categories/show-and-tell	社区自荐项目
本白皮书引用链	第 4/5/10 章案例中的仓库链接	经实机验证的插件

技巧： `topic:dsh-plugin` 是最精准的过滤方式。2026-08-13 时点结果仅个位数——正是入场机会。

7.7 参与生态的完整流程

把 7.3 节的“四步走”展开为**可操作的决策树**：

用起来 → 提 Issue 反馈 → 发插件（npm publish） → 写内容

npm publish 流程简述：

参考 CONTRIBUTING.md：案例需含“场景 / 命令 / 耗时或产物 / 验证方式”——发插件时附上这些信息，转化率更高。

7.8 生态里程碑时间线

时间	事件	意义
2026-08-13	dsh 开源（零日）	MIT 协议发布，Agent 可编程时代起点
2026-08-13	60+ 官方包同步放出	packages/ 下工具/上下文/会话/子代理/MCP/工作流/安全/模型/界面全覆盖
2026-08-13 当天	社区插件爆发	官方讨论区单日 30+ 帖，含插

		件踩坑、TUI 示例、Windows 路径 bug 等
2026-08-13 当周	本白皮书发布	11 章中文教程 + Benchmark + 插件模板，补零日文档缺口
进行中	官方讨论区持续活跃	每周 2-3 帖响应，已覆盖插件 / 路径 / TUI / vision 等方向（见 7.9 节）
规划中	插件模板扩展 / 视频教程 / CI 校验	ROADMAP.md P1 优先级

关键认知：2026-08-13 不是"发布日"而是"生态零日"——官方包、社区插件、中文教程同一天就位。对参与者而言，**现在入场 = 和官方基建同步成长。**

7.9 与官方讨论区的联动

本白皮书已在官方 Discussions 响应 5 帖，以下是**有效参与的实例**：

帖子	主题	响应方式	可复用模式
[#380](https://github.com/deepseek-ai/deepseek-harness/discussions/380)	插件踩坑	结论整理进第 3 章 3.5 节"常见坑"	问题 → 复现 → 写教程 → 回帖引用
[#392](https://github.com/deepseek-ai/deepseek-harness/discussions/392)	TUI examples	补进 docs/config-reference.md	官方缺示例 → 跑通 → 写参考 → 分享链接
[#401](https://github.com/deepseek-ai/deepseek-harness/discussions/401)	Windows 路径 bug	记录进第 12 章跨平台短板	平台 bug → 记录边界 → 帮官方分流
[#384](https://github.com/deepseek-ai/	visionDS	更新能力矩阵中 vision 支持状态	新能力 → 调研 → 更新表格 → 确认

deepseek-harness/ discussions/384)			
[#118](https:// github.com/ deepseek-ai/ deepseek-harness/ discussions/118)	通用讨论	沉淀进 FAQ (docs/faq.md 六类 问答)	高频问答 → 分类沉淀 → 给链接

有效参与的 3 个原则：① 先自己跑通再回帖（带环境信息和报错日志）；② 把结论写成可引用内容（单条回复会沉底，写成章节/FAQ 才能持续产生价值）；③ 链接代替重复（回帖给链接，如"详见第 7 章 7.5 节"）。

结语

dsh 是 2026-08-13 才开源的项目——**生态的每一天都是"早期"**。白皮书会随 dsh 演进持续更新。如果某章命令失效，大概率是 rc 版本迭代所致——以官方 changelog 为准。

祝你在这个全新的生态里，抢到自己的位置。🚀

动手练习（检验你是否真懂了）

- 自查：参考本章 7.1 节官方入口表格
- 自查：参考本章 7.1 节"但鼓励"段落
- 自查：参考本章 7.2 节"发现插件"段落 + 7.6 节搜索渠道表格
- 自查：参考本章 7.5 节两处改动的完整代码片段
- 自查：参考本章 7.3 节第 2 步 + 第 5 章案例的 PR 范式
- 自查：参考本章 7.8 节时间线表格 + 7.9 节联动模式

常见疑问 FAQ

Q1: 官方说"不接受外部 PR", 那我的插件代码放哪? 放在你自己的 GitHub 仓库, 作为独立的 dsh-plugin 生态项目。官方鼓励这种方式——通过插件扩展能力, 无需改 dsh 核心。给仓库加 `dsh-plugin` topic, npm 发布即可。

Q2: Discord 和 Discussions 有什么区别? Discord 适合实时讨论; Discussions 适合正式提案、功能建议、求助 (有记录可查)。官方当前建议的贡献入口是 Discussions。

Q3: 我想写中文教程/博客, 有格式要求吗? 没有官方格式要求。建议包含安装步骤、代码示例、实机截图/日志。可参考本白皮书结构 (TL;DR + 分步讲解 + 练习)。

Q4: 生态零日是什么意思? 为什么说是"先发优势"? 生态零日 = 生态刚起步, 内容/插件/教程几乎为零。先发优势 = 早期入场者容易成为"某个方向的标杆"。类比: 2015 年写 React 教程的人、2018 年做 VS Code 插件的人。

Q5: rc 阶段迭代快, 我的插件会不会刚写完就过时? 有可能。降低风险: ① 依赖用 `^0.1.0-rc.6` 线; ② 关注官方 changelog; ③ 逻辑用纯函数隔离, 接入层薄一点, 升级时只改接入层。

Q6: 我想做 dsh 生态项目, 但从哪开始? 从"自己的痛点"开始。比如: ① 想要 TUI → 做 `tui profile` 插件; ② 想要 token 追踪 → 做 `cost-tracker` 插件; ③ 想要 diff 视图 → 做 `diff` 插件。参考本章 7.4 节阅读路径。

Q7: 7.5 节的 ``link:`` 路径在 Windows 和 macOS 下写法一样吗? 不一样。Windows 用反斜杠 (`link:C:\path\to\plugin`), macOS/Linux 用正斜杠 (`link:/path/to/plugin`)。建议社区插件最终走 npm 发布, 消除路径差异。

Q8: 官方讨论区发帖有什么技巧, 能让官方更快响应? 参考 7.9 节 3 个原则: 带环境信息、带复现步骤、先搜索避免重复。功能建议说明"解决什么问题"比"怎么实现"更重要。

第 8 章：工具与上下文系统

本章目标：理解 dsh 的"能力引擎"——模型能调用哪些工具、上下文是怎么喂给模型的、以及长对话怎么处理。这是从"能跑"到"跑得明白"的关键一章。

TL;DR（本章核心，30 秒版）

8.1 官方能力包地图（60+ 包一览）

dsh 的能力全部以包形式提供（`packages/<group>/<name>`）。新手最需要认识的：

能力域	官方包	作用
工具 (tools)	<code>fs/tool-fs</code> 、 <code>fs/tool-fs-search</code> 、 <code>fs/tool-str-replace-editor</code> 、 <code>shell/tool-bash</code> 、 <code>web</code> 、 <code>skill</code> 、 <code>todo</code>	文件/终端/网页/技能/待办等可调用工具
上下文 (context)	<code>context/*</code> 、 <code>compaction/*</code>	请求上下文组装、长对话压缩
会话 (session)	<code>session/*</code>	持久化、标题、遥测
子代理 (subagent)	<code>subagent/*</code>	委派子任务
MCP	<code>mcp/*</code>	MCP 客户端（外部工具服务器）
工作流 (workflow)	<code>workflow/*</code>	多步工作流编排
安全 (safety)	<code>sandbox/*</code> 、 <code>guard/*</code> 、 <code>interaction/*</code>	沙箱、循环卫生、权限/审批
模型 (llm)	<code>llm/*</code> 、 <code>llm-deepseek</code> 、 <code>llm-retry</code>	模型接入、重试
技能 (skill)	<code>skill/*</code>	技能提供者注册表
界面 (client)	<code>client/*</code> (<code>ui-conversation</code> 、 <code>ui-tool...</code>)	Web UI 各部件

完整清单见官方仓库 `packages/AGENTS.md`。

8.2 内置工具（实测观察）

模型实际可调用的工具名是**简短动词**（实测记录，来自 dsh web 会话与 agent/request 日志）：

工具名	作用	备注
read	读文件	fs 能力
write	写文件	fs 能力
grep	内容搜索	fs-search
glob	文件模式匹配	fs-search
edit / str_replace_editor	精准编辑	工具结果含 locations（用于产物追踪）
bash / pwsh	执行命令	沙箱隔离
todo	待办管理	长任务规划
skill	技能调用	skill-catalog 注入

工具结果与产物追踪（重要概念）：工具的返回里带有 `locations`（文件路径），dsh 用它们做“产物文件行”（对话结尾的产物 chips 就是从这里来的）——**模型改了什么文件，UI 能直接看到并可打开。**

8.3 上下文是怎么喂给模型的

一次模型请求的上下文 = 系统提示 + 技能目录 + 对话历史 + 工具结果。实测在会话日志中可见：

上下文注入 @deepseek-ai/dsh-system-prompt	← 官方系统提示
上下文注入 skill-catalog	← 技能目录

dsh 的上下文机制：

8.4 长对话：compaction（压缩）

长对话会撑爆上下文。dsh 的 `compaction` 插件（如 `compaction-basic`）负责：

对新手：**知道"长对话会自动压缩"即可**，细节是进阶话题。生产环境注意：压缩会丢细节，重要上下文建议手动写进提示词。

8.5 权限与安全模型（了解即可）

安全配置的深度话题超出本白皮书范围；核心认知：**dsh 的工具执行默认有隔离与审批层**，不是裸执行。

8.6 新手最该记住的三件事

动手练习（检验你是否真懂了）

自查：参考本章 8.2 节工具表格

自查：参考本章 8.2 节"工具结果与产物追踪"段落

自查：参考本章 8.2 节"产物文件行"说明

自查：参考本章 8.4 节"长对话自动压缩"段落

自查：参考本章 8.4 节"压缩会丢细节"警告

自查：参考本章 8.5 节"交互审批" + `interaction/*` 包说明

常见疑问 FAQ

Q1：工具名是 ``read`/`write`` 这些简短动词，我写提示词时也要用这些名字吗？ 不需要。你写"读一下这个文件""搜索包含 `foo` 的行"，模型会自动映射到对应工具。简短动词名是插件内部实现细节，提示词用自然语言即可。

Q2: ``edit`` 和 ``str_replace_editor`` 是同一个工具吗? 是。`str_replace_editor` 是内部包名, 模型调用时可能显示为 `edit`。功能一样: 精准替换文件中的某段文本。工具结果里的 `locations` 用于产物追踪。

Q3: `compaction` 压缩后, 模型还能记得之前对话的内容吗? 记得"摘要", 但会丢细节。`compaction` 会修剪历史 + 可选生成摘要, 关键信息保留, 细节丢失。所以重要上下文 (如项目约束、特殊要求) 建议写进系统提示或提示词, 不要只依赖对话历史。

Q4: ``context/*`` 和 ``compaction/*`` 有什么区别? `context/*` 负责"组装每次请求的上下文" (系统提示 + 技能目录 + 对话历史 + 工具 schema)。 `compaction/*` 负责"长对话压缩" (检测溢出 → 修剪 → 摘要)。一个是"装什么", 一个是"装不下时怎么压缩"。

Q5: 安全模型里"沙箱"和"交互审批"有什么区别? 沙箱 (`sandbox/*`) 隔离命令执行环境 (如 `pwsh` 沙箱有 ACL 约束, 限制文件访问)。交互审批 (`interaction/*`) 是"危险操作前要求用户确认" (如删除文件、执行未知命令)。一个管"在哪执行", 一个管"能不能执行"。

Q6: 我想给 `dsh` 加一个新工具 (比如"查数据库"), 应该怎么做? 两种方式: ① 写一个 `host` 插件, 用 `ctx.provide` 注册工具服务 (参考第 3 章 3.4 节扩展点); ② 用 MCP 接入外部工具服务器 (第 9 章 9.1 节)。前者适合深度集成, 后者适合快速接入现有系统。

下一章: [第 9 章: MCP、子代理与工作流](./09-mcp-subagent-workflow.md)

第 9 章：MCP、子代理与 workflow

本章目标：了解 dsh 的三项扩展能力——MCP（接入外部工具）、子代理（并行任务）、workflow（多步编排）。**这些是把 dsh 从"单 Agent"升级为"Agent 系统"的开关。**

△ 本章基于官方架构文档（`packages/AGENTS.md`）与包结构整理；部分功能示例标注「待实测」——rc 版本迭代快，具体用法以官方 changelog 为准。

TL;DR（本章核心，30 秒版）

9.1 MCP：接入外部工具生态

MCP (Model Context Protocol) 是"给 Agent 插外部工具"的开放协议。dsh 提供 mcp 客户端包（`@deepseek-ai/dsh-mcp-client`）。

能做什么：通过 MCP 接入任何 MCP 服务器（数据库、浏览器、图表、公司内部系统.....）——比如社区已出现的 `dsh-plugin-cost-tracker`（追踪 token）就是 MCP/插件形态。

新手定位：MCP 是"生态扩展"，等你在 dsh 里有了明确的工具缺口再来接。先跑通内置工具，再加 MCP。

9.2 子代理 (subagent)：并行干活

是什么：把任务委派给子 Agent 并行执行（`packages/subagent/*`）。

典型场景：

对新手：子代理是"高阶武器"——先用好单 Agent，理解任务分解后再上子代理。错误示范是"什么任务都开子代理"，反而增加协调开销。

9.3 workflow (workflow)：多步编排

是什么：`packages/workflow/*` 提供多步 workflow 编排（worker-thread provider + tool Consumer）。

和"多轮对话"的区别：普通对话是"模型自由发挥"， workflow 是"把步骤定义成流程，按顺序/条件执行"——适合**确定性流程**（如：拉取数据 → 清洗 → 生成报表 → 校验）。

官方示例（`packages/examples/`）：有可运行的 `cordis.yml` workflow 示例。

9.4 组合起来：一个"Agent 系统"长什么样

```

你的提示词（目标）
↓
父 Agent（规划）
├─ 子代理 A：调研模块 A（并行）
├─ 子代理 B：调研模块 B（并行）
└─ 工具链：read/grep/bash + MCP（数据库）
↓
Workflow（如：校验 → 汇总 → 输出报告）
↓
产物（可打开/追踪）

```

新手路径建议：

9.5 社区生态示例（2026-08-13 快照）

官方 Discussion 里已出现的社区项目：

生态正在快速生长——**现在是进场搭积木的好时候。**

9.6 MCP 接入配置示例

以下给出在 `cordis.patch.yml` 中挂载 MCP 客户端的**配置片段**（语法与第 3 章 3.2 节一致，以社区 token 追踪插件为例）：

```

- insert:
  - id: mcp-client
    name: '@deepseek-ai/dsh-mcp-client'
    config:
      servers:
        - name: cost-tracker
          command: npx
          args: ['-y', 'dsh-plugin-cost-tracker']
          env:

```

```
DSH_API_KEY: '${DSH_API_KEY}'
```

字段说明：id (cordis 链标识)、name (npm 包名)、config.servers (服务器列表，每服务器独立进程)、command/args (启动命令与参数)、env (环境变量，支持 \${VAR} 引用宿主变量)。

△ 上述配置基于 packages/mcp/ 目录结构与 cordis 挂载语法推断，具体字段以官方文档为准 (rc 阶段可能变动)。验证思路 (推断，待实测)：挂载后重启 dsh web，发涉及 token 计数的任务，观察日志是否出现 MCP 调用记录。

9.7 子代理配置示例

并行子代理通过 packages/subagent/* 提供。以下是在 cordis.patch.yml 中启用子代理能力的配置片段：

```
- insert:
  - id: subagent
    name: '@deepseek-ai/dsh-subagent'
    config:
      maxConcurrency: 3
      timeout: 120000
```

适用场景与并发策略：

场景	子代理数	父代理职责	子代理职责
大仓库多模块调研	3-5 个	划分模块边界、汇总结论	各模块独立调研
长任务分解	2-3 个	拆分子任务、校验完整性	各子任务独立执行
独立验证	1-2 个	提供结论与验证标准	交叉检查
多文件并行编辑	2-4 个	锁定文件范围、避免冲突	各子代理写不同文件

⚠️ `maxConcurrency` 与 `timeout` 基于包名与 `cordis` 惯例推断（推断，待实测）。上下文隔离机制需查阅官方 `packages/subagent/README.md`。

9.8 工作流 YAML 示例

以下是一个**确定性工作流**的 `cordis.patch.yml` 片段，模拟"数据拉取 → 清洗 → 报表 → 校验"四步（参考第 10 章案例 A 的任务结构）：

```
- insert:
  - id: workflow
    name: '@deepseek-ai/dsh-workflow'
    config:
      steps:
        - id: fetch
          tool: bash
          command: 'curl -o raw.json https://api.example.com/data'
        - id: clean
          tool: bash
          command: 'python clean.py raw.json cleaned.json'
          dependsOn: [fetch]
        - id: report
          tool: bash
          command: 'python visualize.py cleaned.json chart.png'
          dependsOn: [clean]
        - id: validate
          tool: bash
          command: 'python verify.py cleaned.json'
          dependsOn: [report]
```

与案例 A 的对应：工作流 `dependsOn` 显式定义顺序，而案例 A 的"多轮对话"由模型自由决定工具。工作流适合"步骤固定"；自由对话适合"探索性"。

⚠️ `steps/dependsOn` 基于 `packages/workflow/` 包名与常见 DSL 推断（推断，待实测）。官方 `packages/examples/` 有可运行示例，建议以其为准。

9.9 三者对比表

维度	MCP	子代理	工作流
适用场景	接入外部系统（数据库	大任务并行分解（调研	确定性流水线（拉取→

	/浏览器/内部 API)	/验证/多模块)	清洗→报表→校验)
复杂度	中（需配置服务器 + 理解协议）	高（需任务分解设计 + 结果汇总）	低-中（YAML 配置，调试依赖图较繁琐）
失败处理	服务器崩溃 → 重连或报错（推断，待实测）	单个子代理失败 → 父代理决定重试/忽略/终止	单步失败 → 可选暂停/跳过/回滚（推断，待实测）
性能开销	额外进程通信开销（stdio/sse）	并发节省墙钟时间，但增加协调开销	串行执行，无额外模型开销，但无并行收益

决策建议：有明确工具缺口 → MCP；任务能拆成互不依赖的几块 → 子代理；步骤固定、需严格按序 → 工作流。三者可组合：子代理并行调研 → 内部用 MCP 查数据库 → 工作流汇总报告。

9.10 踩坑补充

#	坑	现象	解法
1	MCP 连接失败	挂载后启动报错或 MCP 工具无响应	检查 <code>command/args</code> ；先手动跑 <code>npx -y xxx</code> 验证（推断，待实测）
2	子代理上下文隔离	子代理看不到父 Agent 历史，导致重复提问	父代理委派时把关键上下文写进任务描述（推断，待实测）
3	工作流单步失败	某步 <code>bash</code> 返回非零，后续步骤继续执行	配置 <code>onError: pause</code> 或每步后加校验脚本（推断，待实测）
4	子代理文件冲突	多个子代理同时写同一个文件	父代理规划时划分文件边界（推断，待实测）
5	MCP 环境变量未注入	<code>\${DSH_API_KEY}</code> 解析为空	确认宿主 <code>shell</code> 已 <code>export</code> ；或写死（仅本地测试）（推断，待实测）

所有标注"推断，待实测"的内容均基于 `packages/` 目录结构与 `cordis` 配置惯例推断，非本白皮书实机验证结果。rc 阶段请以官方文档为准。

9.11 四阶段新手路径（含验收标准）

把 9.4 节的"四阶段"扩展为**每阶段有明确验收标准**的升级路径：

阶段	能力	验收标准（怎么知道自己该进入下一阶段）	参考章节
阶段一	单 Agent + 内置工具	能独立完成：创建文件 → 搜索 → 编辑 → 运行验证，全程不卡壳	第 2-5 章
阶段二	+ MCP	成功挂载一个 MCP 服务器，任务中调用其工具且日志证明调用发生	9.1 节 + 9.6 节
阶段三	+ 子代理	成功发起一次并行子代理任务（如同时调研 2 个模块），父代理能汇总结果	9.2 节 + 9.7 节
阶段四	+ 工作流	成功配置并执行一个 3 步以上工作流，每步按 dependsOn 顺序执行	9.3 节 + 9.8 节

为什么不要跳级：阶段一没跑通 → 阶段二报错无法判断是配置错还是基础工具链不会用；阶段二没跑通 → 阶段三并行放大问题；阶段三没跑通 → 阶段四 YAML 写出来也是错的。阶段一到阶段二之间，先读第 8 章 8.1 节能力包地图——确认需求是否已内置，避免为"已有能力"配 MCP。

动手练习（检验你是否真懂了）

常见疑问 FAQ

Q1：MCP 是什么？和 dsh 插件有什么区别？ MCP 是"给 Agent 插外部工具"的开放协议。插件在 dsh 运行时内注册能力（host 半跑在 Node 进程）；MCP 服务器是独立进程，通过协议通信。插件深度集成、性能好；MCP 解耦、可复用、适合接入现有系统。

Q2：子代理是怎么"并行"的？会不会有冲突？ 子代理由父 Agent 委派，各自独立执行（`packages/subagent/*`）。并行时各子代理有自己的上下文，不会互相干扰。但多个子代理改同一个文件可能冲突——建议任务分解时避免交叉。

Q3：工作流的"确定性流程"是什么意思？和模型自由发挥有什么区别？ 工作流把步骤定义成流程（如 `cordis.yml`），按顺序/条件执行，每步做什么是指定的。多轮对话是"模型自由发挥"。确定性流程适合"必须按固定步骤走"的场景，自由发挥适合"探索性任务"。

Q4：我想接一个 MCP 服务器（比如数据库），具体怎么操作？（以官方 changelog 为准）① 安装 `@deepseek-ai/dsh-mcp-client`；② 在 `cordis.patch.yml` 挂载（参考 9.6 节）；③ 配置服务器地址/命令；④ 重启 `dsh web`。详见官方 `packages/mcp/` 文档。

Q5：子代理和"开多个 dsh 进程"有什么区别？ 子代理是 `dsh` 内部委派机制（`packages/subagent/*`），共享父 Agent 上下文和会话。多进程是完全独立的会话。子代理适合"任务分解后并行执行 + 结果汇总"，多进程适合"完全独立的任务"。

Q6：本章说部分功能"待实测"，我怎么知道哪些功能已经可用？ 查官方 `packages/AGENTS.md` 和 `packages/` 目录。有完整源码 + 示例的功能基本可用；只有包名没有文档的可能还在开发中。rc 阶段以官方 changelog 为准。

附录 A： [术语表与命令速查](./appendix-glossary.md)

第 10 章：复杂实战案例（dsh 真实跑出来的）

本章目标：用两个**真实在 dsh 里跑完的复杂任务**，展示 dsh 在多步工具链下的实际能力、产出质量与耗时。 **隐私声明**：两个案例全部使用**合成数据/自造代码**，不涉及任何真实业务数据或敏感信息。

TL;DR（本章核心，30 秒版）

案例 A：数据质量分析 + 清洗 + 可视化（186 秒）

任务

给 dsh 一份含脏数据的合成 JSON（52 行：缺失值、类型错误、重复行、异常值 99999），
要求：

dsh 怎么做的（工具链）

```
read(sales_data.json) → write(clean.py) → bash(python clean.py)
→ write(visualize.py) → bash(python visualize.py) → read(输出) → 总结
```

产出与验证

产物	结果
clean.py	✅ 运行成功，52 → 35 行 ，所有问题归零
visualize.py + chart.png	✅ 生成有效 PNG（1950×825）
清洗策略	缺失金额中位数填充、异常值删除、去重、类型统一

[图: dsh 生成的销售额可视化（案例 A 产物）]

输出要点（体现判断力，不是机械执行）

dsh 不仅执行了任务，还主动说明了数据处理的权衡与假设——这是 agent 工程层"会思考"的证据。

案例 B：5-bug 代码修复 + 49 个测试（94 秒）

任务

给 dsh 一个**故意埋了 5 个 bug** 的 Python 计算器模块，要求：找到所有 bug → 修复 → 编写完整单元测试 → 运行验证。

埋的 bug

dsh 的修复 + 测试（验证：49 passed）

完整测试见 [case-test-calculator.py](./assets/case-test-calculator.py)——49 个用例覆盖：

函数	覆盖
add	正/负/零/浮点/精度/交换律
subtract	负结果/零边界/浮点
multiply	符号组合/乘零/浮点
divide	整除/符号/浮点/ 除零必须抛 ZeroDivisionError
factorial	0!/1! 边界/已知值/ 负数必须抛 ValueError
format_money	补零/四舍五入/ 0.1+0.2 舍入 /负数

```
python -m pytest test_calculator.py -v # 49 passed in 0.37s
```

观察：dsh 的表现

案例总结：dsh 在复杂任务上的画像

维度	表现
多步工具链	✅ 自动编排（读→写→跑→验证→总结）
复杂任务耗时	94s-186s（同模型同网关，比 benchmark 简单任务慢但可控）
产出质量	✅ 有判断力（数据权衡说明、测试边界设计）
产物追踪	✅ 生成文件可在对话末尾直接打开

失败恢复	本案例未触发；生产建议配 guard 超时 + 人工审批
------	------------------------------

给新手的启示：dsh 处理"多文件、多步骤、要验证"的任务很顺手——这也是它作为 Agent 运行时的主要价值场景。复杂任务建议：提示词把**验收标准写清楚**（如"运行验证"），dsh 会照着闭环。

动手练习（检验你是否真懂了）

- 自查：参考本章"案例 A"的"dsh 怎么做的"段落
- 自查：参考本章"案例 B"的测试覆盖表格
- 自查：参考本章案例 A 的任务描述 + 产出验证表格
- 自查：参考本章案例 B 的任务描述 + "dsh 的表现"观察段落
- 自查：参考本章案例 A 的"输出要点"段落 + 案例总结表格
- 自查：参考本章"给新手的启示"段落 + 案例 A 的工具链（含 bash 运行验证）

常见疑问 FAQ

- Q1：案例 A 耗时 186 秒，案例 B 耗时 94 秒，这个速度算快还是慢？** 取决于任务复杂度和模型档位。同模型同网关下，比 benchmark 简单任务慢但可控。关键不是绝对速度，而是"多步工具链自动编排 + 产出质量 + 无人工干预"的综合价值。如果想提速，参考第 6 章的推理档位策略 + tool-turbo 插件。
- Q2：案例里的"合成数据"是什么意思？为什么不用真实数据？** 隐私保护。本白皮书不涉及任何真实业务数据或敏感信息。合成数据是"故意构造的、不含真实信息的数据"，用于演示 dsh 的能力。你在自己环境里可以用真实数据，但要注意数据安全。

Q3: 如果 dsh 在复杂任务中"卡住了" (比如某步工具调用失败), 怎么办? 几种处理: ① 看 dsh 进程日志, 定位哪步失败; ② 检查工具调用的输入是否正确 (如文件路径、命令语法); ③ 配 guard 超时 (guard/* 包), 避免无限等待; ④ 生产环境建议配人工审批 (interaction/*), 危险操作前确认。

Q4: 案例 B 的 49 个测试是 dsh 自动生成的, 质量怎么样? 质量不错。覆盖了边界 (除零/负数/精度)、异常 (必须抛特定异常)、常规 (正/负/零/浮点)。但自动生成的测试可能漏掉"业务逻辑特有的边界"——建议人工 review 测试设计, 补充领域特定的用例。

Q5: 我想让 dsh 跑一个"多文件、多步骤"的任务, 提示词怎么写效果最好? 三个原则: ① 明确目标 ("做什么"); ② 明确验收标准 ("怎么算完成", 如"运行验证"49 个测试全过); ③ 明确约束 ("不能做什么", 如"不能删原始数据")。参考本章两个案例的提示词结构。

Q6: 案例总结说"失败恢复本案例未触发", 那 dsh 的失败恢复能力怎么样? dsh 有 compaction (上下文溢出恢复)、guard (超时)、interaction (审批) 等机制, 但复杂场景的失败恢复还在演进中。生产环境建议: ① 配 guard 超时避免卡死; ② 配人工审批避免误操作; ③ 长任务分步验证, 不要"一口气跑完再检查"。具体配置参考第 8 章 8.5 节安全模型。

附录 A: [术语表与命令速查](./appendix-glossary.md)

扩展案例 (第二批, 三个功能领域)

完整执行报告见 [case-expansion-report.md](./case-expansion-report.md), 产物在 docs/assets/。全部合成数据。

案例 C: HTML 解析 (Web 前端, 约 3m18s)

任务: 给定含表格+表单+图表的合成 HTML, 编写零第三方依赖的解析脚本提取表格并输出 CSV。

dsh 的表现:

产物: [case-c-parser.py](./assets/case-c-parser.py) | 验证: 输出 9 行×7 列 CSV, 与源表逐字段一致

案例 D：代码重构（软件工程，约 4m37s）

任务: 把约 200 行过程式订单脚本（含重复代码 `calc_order_total / calc_order_total_v2`）重构为 OOP + 职责分离，行为一致，测试验证。

dsh 的表现：

产物: [case-d-orders_refactored.py](./assets/case-d-orders_refactored.py) + [case-d-test_refactor.py](./assets/case-d-test_refactor.py) | 验证: **17/17 PASS**

案例 E：API 文档生成（文档，约 1m04s）

任务: 给含 8 个函数的 Python 模块生成完整 Markdown API 文档（参数表/返回值/异常/示例/边界）。

dsh 的表现：

产物: [case-e-api.md](./assets/case-e-api.md)

第二批案例画像

维度	表现
跨领域能力	✅ Web/工程/文档三种任务都能闭环
工程意识	✅ 零依赖权衡、沙箱感知、向后兼容、契约差异识别
验证习惯	✅ 每个案例都运行验证（含脚本级产物对比）
耗时	1-5 分钟/案例，复杂度越高越久

总结: dsh 在"要写代码 + 要验证"的工程任务上表现稳定，且**会做权衡（依赖/兼容/安全）**——这是 agent 从"能答"到"能干活"的分水岭。

第 11 章：未来展望——dsh 与 Agent 生态的演进预测

本章目标：从多个角度推演 dsh 及其生态的未来——**技术、生态、竞争、行业、开发者机会、风险**。预测不是断言，是"基于当前事实的推演"，供你判断是否值得投入。

TL;DR（本章核心，30 秒版）

11.1 技术演进预测

时间	预测	依据
短期	0.1.0 正式版发布（去掉 rc），破坏性变更收敛	当前 rc.6，官方明示"将有破坏性变更"，迭代极快
短期	官方补上 TUI 与交互式 CLI（社区呼声最高，#172 15+ 评论）	官方 Discussion 最热需求
中期	缓存/性能优化成为主旋律（reasoning_effort 精细化、缓存命中率工具化）	高缓存命中率已是成本优势（第 5 章），会被工具化显性化
中期	插件生态标准化（插件市场/一键安装/质量分级）	官方已有 dsh-plugin topic，插件数量增长必然催生市场
长期	Agent 运行时架构沉淀为"工程范式"（类似当年 Linux 内核模式的 Agent 版）	"everything is a plugin" 已被验证为可组合架构

11.2 生态发展预测

11.3 竞争格局预测

维度	dsh 的定位演进
vs Claude Code	短期仍"开箱即用不如"，长期靠"可定制 + 开源 + 成本"差异化
vs OpenCode	都开源；dsh 的官方后端 + 插件生态是差异点（OpenCode 无官方后端）
vs 自建框架	dsh 是"半成品到成品的桥梁"——比自建省事，比闭源可控

模型侧	DeepSeek 模型每代迭代都会放大 dsh 生态价值 (模型 + 运行时协同)
-----	---

关键判断：dsh 的胜负手不在"模型多强"（那是 DeepSeek 的事），而在"**Agent 工程层能否成为标准**"——谁能定义"插件怎么写、生态怎么长"，谁就赢下这一层。

11.4 行业应用预测

11.5 开发者机会（现在入场的人）

角色	机会	时机
教程/内容作者	中文教程真空（白皮书是第一份）	现在（红利最大）
插件开发者	官方缺位方向：TUI、远程访问、移动端、缓存工具	现在
工具型项目	提速、耗时可视化、MCP 工具包	现在（我们已做 tool-turbo 验证）
生态基础设施	插件市场、模板生成器、评测工具	中期
行业解决方案	金融/教育等行业插件包	中期

11.6 风险与不确定性（诚实版）

11.7 白皮书自己的展望

11.8 给读者的最终建议

预测会错，但"现在入场的成本最低"这一点，大概率不会错。

附录 A：[术语表与命令速查](./appendix-glossary.md)

第 12 章：已知不足与边界——诚实版

本章目标：不吹不黑，列出 dsh 当前的**已知不足与使用边界**。写这本白皮书时 dsh 仍处于 0.1.0-rc.6（预发布阶段），这些不足是"此刻的事实"，不是"永远的宿命"——它们会随版本迭代变化。

本章所有"已知问题"均来自本机实测 + 官方仓库 deepseek-ai/deepseek-harness 讨论区（截至 2026-08-13）的公开反馈。

TL;DR（本章核心，30 秒版）

12.1 rc 阶段的不稳定性

破坏性变更随时发生

dsh 从 rc.1 到 rc.6 已经历过至少一次**依赖断裂**：早期版本 @deepseek-ai/* 包的版本线不一致会导致插件装不上（404/ERR_MODULE_NOT_FOUND）。rc.6 收敛了很多，但**没有任何承诺**说 rc.7/正式版不会再来一次。

对你的实际影响：

[!WARNING] 生产环境请谨慎评估。白皮书写作时官方仍在快速迭代，**今日的写法明天可能就废弃**。

已知的 rc 坑（本机/社区复现）

坑	表现	状态
link 开发依赖断裂	@deepseek-ai/* 本地 link 时报 ERR_MODULE_NOT_FOUND , npm 安装却正常	已定位（扁平兜底目录机制）， 需锁依赖线规避
Windows 路径 0x00	含中文/特殊字符路径 workspace 创建失败	社区反馈中（#151/#396）

	(ENOENT)	
fs-sandbox 竞态	post-check 路径检查与 workspace-write 存在时序竞态	社区反馈中 (#159)
TUI 缺失	官方 examples 无交互终端 UI (社区自建有, 未纳入)	建议中 (#392)

12.2 插件生态早期

现状：官方包 60+，但生态整体处于"零日"状态。

判断：生态玩法现在入场是早期红利（竞争少、官方扶持意愿强），但**不要指望"照着官方文档就能跑"**——踩坑是常态。

12.3 跨平台短板

dsh 在 macOS/Linux 上表现成熟，但 Windows 支持有明显的**第二公民感**：

12.4 性能边界未充分验证

白皮书的 benchmark 覆盖了**单 agent、常规任务规模**，以下场景缺少公开数据：

12.5 与成熟 Agent 的差距

对照 Claude Code、Codex 等成熟产品，dsh 的差距是**结构性的**（不是补几个功能就能追上）：

维度	dsh (rc.6)	成熟 Agent
生态阶段	零日，第三方资产稀少	成熟，生态完整
开箱即用	需理解 profile/插件体系	装完就跑
稳定性	rc 期，破坏性变更风险	稳定版，长期兼容承诺
文档	官方偏架构，上手靠社区	官方文档完善 + 教程丰富
IDE/工具链集成	起步	深度集成

dsh 的定位：不是"开箱即用的整车"，是"可编程的乐高底座"。选择 dsh = 选择**自由度**，代价是**自己要动手拼**。

12.6 官方策略风险

作为选型者，必须考虑的黑天鹅：

12.7 白皮书自身的局限

最后，诚实说这本白皮书自己：

使用建议：把本白皮书当"rc.6 快照 + 方法论"，不要当"永恒圣经"。版本更新时，先跑一遍 [roadmap 学习路径](./roadmap.md) 的验收标准，再决定要不要更新你的依赖线。

本章信息截至 2026-08-13 (dsh 0.1.0-rc.6) 。后续版本如有更新，欢迎提 Issue/PR 修正。

附录 A：术语表与命令速查

术语表

术语	一句话解释
Harness	套在模型外的工程层：会话、工具、上下文、循环控制
dsh	DeepSeek Harness 的命令行名（dsh 命令）
Profile	一种可启动形态（web / headless / 自定义）， = bundle 栈 + patch 层
Bundle	一组插件的集合（官方：dsh-base、dsh-web-app、dsh-headless）
Cordis	dsh 底层的插件容器（依赖注入、事件、生命周期）
插件 (Plugin)	cordis 插件；可同时携带 host 半（Node）与 client 半（浏览器）
host 半 / client 半	插件在 Node 侧（工具/服务/事件）与浏览器侧（UI）的两副面孔
扩展点	官方提供的钩子（agent/request、settings、conversationEvents、slots）
agent/request waterfall	每步模型请求前可改配置的事件链（插件在此注入 reasoningEffort 等）
reasoning_effort	思考强度（low/high/max）——速度与质量的权衡旋钮
headless	一次性 CLI 任务模式（dsh --profile headless "任务"）
compaction	长对话上下文压缩
subagent	子代理（并行委派任务）
MCP	模型上下文协议（接入外部工具服务器）
workflow	多步确定性工作流编排
locations	工具返回的文件路径（驱动物物追踪）
extension point	官方钩子（agent/request 等），插件接入点
waterfall	事件链：监听者可改配置传给下一个（agent/

	request 用此注入)
reasoning_effort	思考强度旋钮 (low/high/max)
context cache	DeepSeek 提示词缓存 (重复输入按折扣价计费)
缓存命中率	输入 token 走缓存价的比例 (dsh 实测可到 97%)
headless	一次性 CLI 任务模式
TUI	终端 UI (官方未内置, 需插件)
turn	对话的一轮 (用户+助手+工具调用)
step	turn 内的一个推理步骤
guard	循环卫生/工具超时插件
skill	技能 (skill-catalog 注入上下文, 模型按需调用)
inject	cordis 依赖注入 (插件声明所需服务)
产物文件	模型创建/修改的文件 (对话末尾的可打开 chips)
sandbox	命令执行的隔离沙箱
reasoning_effort	同「思考强度」
rc	预发布版本 (0.1.0-rc.6) , 迭代快、可能有破坏性变更

命令速查

dsh 核心

```
dsh web                # 启动 Web UI
dsh --profile headless "任务"  # 一次性任务 (脚本/CI)
dsh --version          # 版本
dsh --dump-config      # 合成配置树
dsh plugin --profile <name> add <pkg>  # 安装插件
```

环境

```
node --version                # 需 ≥ 22
npm install -g @deepseek-ai/dsh # 全局安装
npx -y @deepseek-ai/dsh web    # 免安装运行
```

排障

```
netstat -ano | findstr 3080    # 端口占用 (Windows)
taskkill /PID <pid> /F         # 杀进程
cat ~/.dsh/settings.yaml      # 全局配置 (模型/推理档位)
```

插件开发

```
# 挂载到 profile (web 为例)
# ① package.json 加依赖: "pkg": "link:C:\\path\\to\\pkg"
# ② cordis.patch.yaml 加:
#   - insert:
#     - id: <插件 id>
#       name: <npm 包名>
cd ~/.dsh/profiles/web && pnpm install    # 安装
```

配置参考 (settings.yaml)

```
agent-default-model:
  model: deepseek-v4-flash    # 或 deepseek-v4-pro
  reasoningEffort: high      # off (关闭思考/最快) / high (默认) / max (最强)
```

附录 B: [同模型多 Agent 实测](./benchmark.md)

附录：同模型 × 不同 Agent 实测对比（Benchmark）

实测日期：2026-08-13 | 模型：deepseek-v4-flash（同一 opencode-go 网关，同一 API key） | Agent：dsh / opencode / omp 目的：**控制模型变量，对比 Agent 工程层的差异**（提示词、工具链、轮次管理）。扩展：2026-08-13 追加 T4（生成 markdown 报告） / T5（多文件小重构），样本 3 轮 → 5 任务 × 3 轮。

方法

项	说明
模型	deepseek-v4-flash（三 agent 均走 opencode-go 网关 + 同一 key，公平对照）
Agent	dsh (headless profile)、opencode (opencode run)、omp (--print 非交互)
任务	T1 创建文件 / T2 搜索统计 / T3 修复 bug + 验证 / T4 读数据生成报告 + 自复核 / T5 多文件重构 + 测试通过
环境	Windows 11 + Node 24，独立工作目录，无预置上下文
度量	耗时（秒）+ 正确性（人工核对产物/输出）

任务说明（T4/T5 为合成数据，无隐私风险）：

任务	输入	要求	人工核对标准
T4 生成报告	data.json（4 条合成销售记录）	生成 report.md：数据表格 + 汇总（总销量/总营收/最畅销），自复核	汇总数字与数据一致（397 / 130853 / 耳机）
T5 多文件重构	mathops.py + 依赖它的主文件 main.py（含断言）	把 multiply 的乘法逻辑提取为 _multiply 辅助函数，不改 main.py，python main.py 通过	出现 _multiply 且 multiply 委托调用、断言全过

结果（3 轮采样，取中位数）

Agent	T1 创建文件	T2 搜索统计	T3 修bug+验证	T4 生成报告	T5 多文件重构	总计（中位）	正确率
omp	10s	11s	15s	21s	13s	70s	45/45
dsh	22s	15s	48s	26s	19s	130s	45/45
opencode	35s	37s	42s	30s	28s	172s	45/45

总计 = 5 任务中位数相加；扩展前（仅 T1-T3）为 omp 36s / dsh 85s / opencode 114s。

[图: benchmark 柱状图]

原始 3 轮（秒）：

Agent	轮次	T1	T2	T3	T4	T5
dsh	1 / 2 / 3	11 / 27 / 22	11 / 15 / 22	27 / 48 / 74	30 / 26 / 25	19 / 25 / 18
opencode	1 / 2 / 3	18 / 35 / 35	19 / 37 / 37	50 / 41 / 42	30 / 49 / 30	28 / 28 / 32
omp	1 / 2 / 3	9 / 10 / 12	9 / 11 / 12	13 / 33 / 15	23 / 21 / 17	16 / 12 / 13

解读

可复现

```
# 三个 agent 的命令（独立目录）
dsh --profile headless "任务"
opencode run "任务" --model opencode-go/deepseek-v4-flash
omp "任务" --model deepseek-v4-flash --print
```

T4/T5 任务原文（T1-T3 见上一版本记录）：

T4: 读取当前目录 `data.json` 中的数据, 生成 markdown 报告 `report.md`: 包含每行数据的表格 (产品/销量/单价三列) 和汇总 (总销量、总营收、最畅销产品), 生成后自己复核汇总数字与 `data.json` 一致, 完成后回复 已生成

T5: 重构 `mathops.py`: 把 `multiply` 函数内的乘法逻辑提取为私有辅助函数 `_multiply(a, b)`, `multiply` 改为调用 `_multiply`; `main.py` 依赖 `mathops.py` 并包含断言测试, 不得改动 `main.py`; 重构后运行 `python main.py` 验证所有断言通过, 完成后回复 已重构

完整任务定义、合成数据与产物: `~/agent-bench/bench-ext/` (T4/T5 每 agent 独立目录); T1-T3 见 `~/agent-bench/`。白皮书仓库内 `benchmark/` 目录规划中。

一页速查卡 (Cheatsheet)

打印/收藏这张卡，日常用 dsh 不用翻书。

安装与启动

```
npx -y @deepseek-ai/dsh --version      # 免安装运行
npm i -g @deepseek-ai/dsh             # 全局安装
dsh web                                # Web UI → http://127.0.0.1:3080
dsh --profile headless "任务"          # 一次性任务 (脚本/CI)
```

配置 (~/.dsh/settings.yaml)

```
agent-default-model:
  model: deepseek-v4-flash      # 或 deepseek-v4-pro
  reasoningEffort: high        # off (关闭思考/最快) / high (默认) / max (最强)
```

推理档位

档位	用途
low	简单/批量/工具链廉价轮 (最快)
high	日常默认
max	复杂推理/长链规划

工具链任务 90% 时间在思考——降档是最快提速。

核心命令

命令	用途
dsh web	Web UI
dsh --profile headless "任务"	一次性任务
dsh --dump-config	看合成配置

dsh plugin --profile <n> add <pkg>	装插件
------------------------------------	-----

插件挂载（两步）

```
# package.json 加依赖
"my-plugin": "link:C:\\path\\to\\my-plugin"
# cordis.patch.yml 加挂载
- insert:
  - id: my-plugin
    name: my-plugin
```

```
cd ~/.dsh/profiles/web && pnpm install && dsh web
```

提示词黄金法则

缓存省钱

排障速查

现象	解法
端口占	netstat -ano findstr 3080 → kill
模型无响应	查 settings.yaml + API key
插件装不上 404	依赖用 ^0.1.0-rc.6 线
长任务崩	全局安装（绕 npx）+ 降推理档

术语（速记）

profile 形态 · bundle 插件组 · host/client 半 服务端/界面 · 扩展点 官方钩子 ·
waterfall 请求链 · compaction 上下文压缩 · headless 一次性 CLI · locations 产物路径

完整教程见 [dsh-handbook 白皮书](<https://github.com/Electricitysheep/dsh-handbook>)。